EPICODE

Cyber Security & Ethical Hacking

Linguaggi di programmazione: Python

Agenda

- Introduzione
- Socket di rete
- Port Scanner
- Rilevatore verbi HTTP
- Esercizi in Python

EPICODE

Introduzione

Perché lo studiamo

Studiare Python in ambito sicurezza informatica è estremamente importante per diverse ragioni.
Analizzare come sono strutturati alcuni tipi di programmi che utilizzeremo ci aiuta a capire come funzionano e, di conseguenza, a comprenderli meglio.

Cosa apprenderai alla fine del modulo

Alla fine del modulo, avrai capito come funzionano alcuni programmi che useremo durante il corso.

Cosa Faremo

Ora che abbiamo visto le basi del Python per poter scrivere dei programmi, vediamo degli esempi di codice che sono molto utili dal punto di vista degli Ethical Hacker, nella fattispecie:

- **Socket di rete.**
- **Port Scanner.**
- **Rilevatore verbi HTTP.**
- **Esercizi in Python**

EPICODE

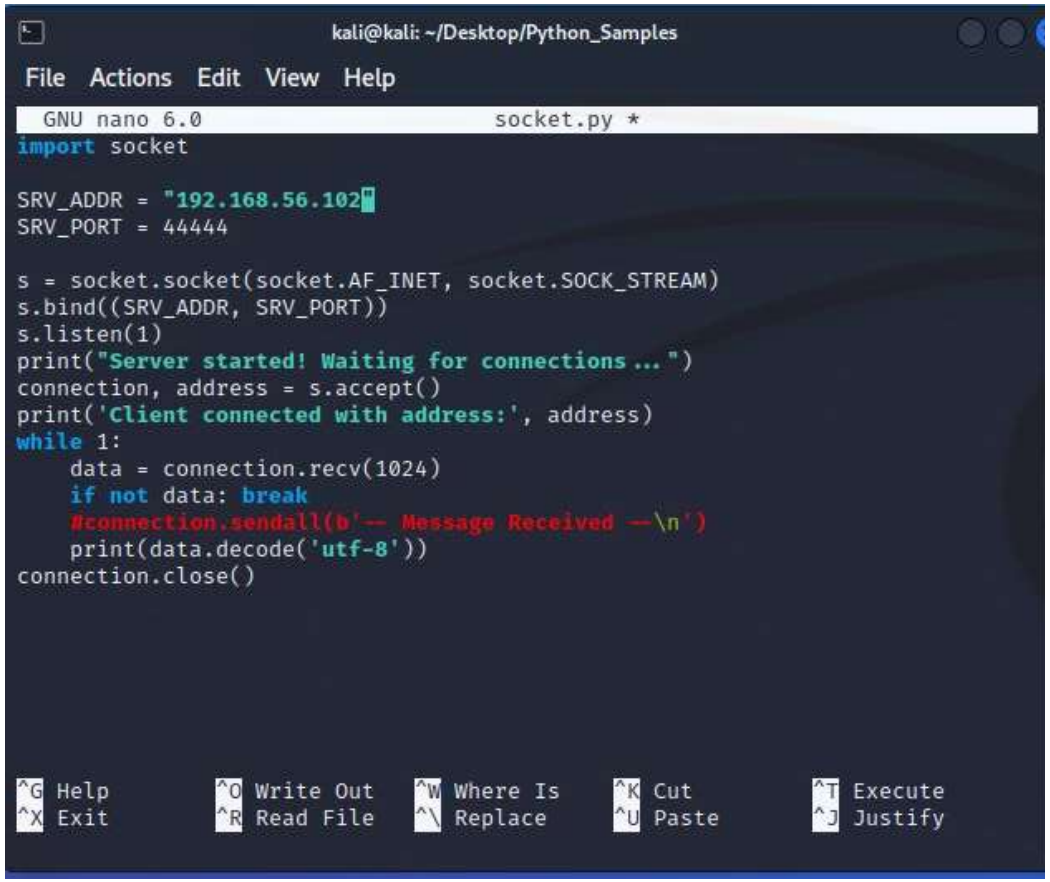
Socket di rete

Socket di Rete

I socket di rete sono utilizzati nelle reti di computer e permettono di scambiare pacchetti (dati) tra due computer, una sorgente ed un destinatario. Quello che andremo a vedere è un programma che resta in ascolto di comunicazioni TCP in entrata, come se fosse un server, su uno specifico indirizzo IP e porta.

1. La prima cosa da fare è importare il modulo sulle socket. Questo ci permetterà di utilizzare tutte le funzioni che vedremo a breve.

2. Vogliamo che il nostro programma resti in ascolto per accettare delle connessioni TCP, ricordando le nozioni di network che abbiamo visto, un servizio è attivo su una coppia IP:porta.
   Creiamo due variabili, una con l'indirizzo IP del PC dove vogliamo mettere il servizio in ascolto, ed una con una porta che poi useremo per la connessione.



Socket di Rete

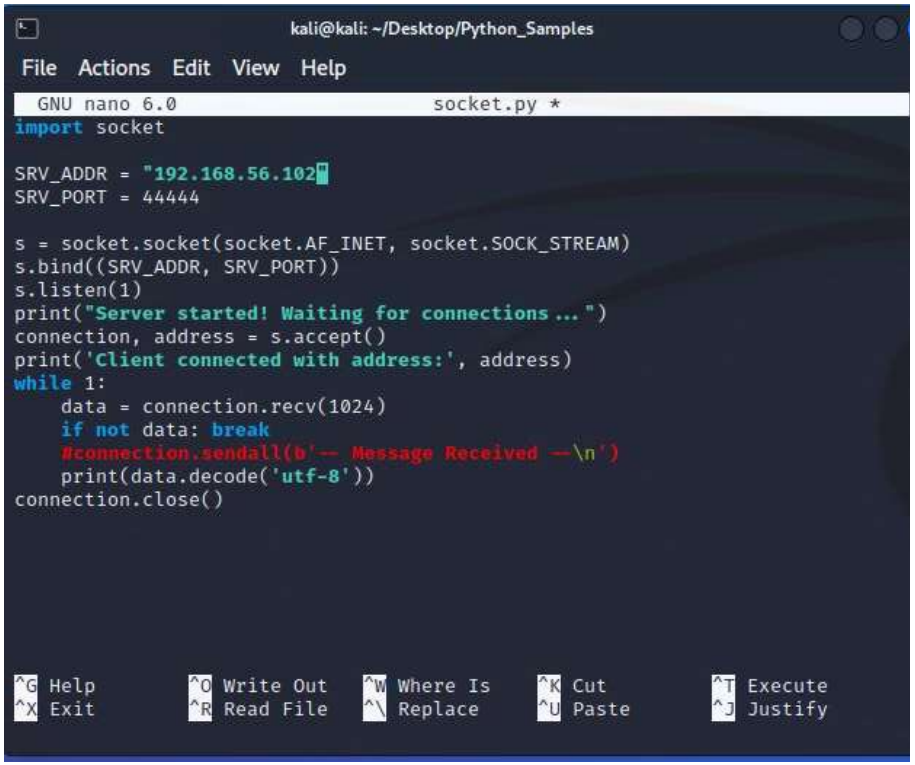
3. Creiamo il socket in ascolto. Per farlo utilizziamo una serie di funzioni della libreria socket:
   socket.socket: crea un nuovo socket (che nell'esempio a fianco abbiamo chiamato «**s**». La funzione accetta dei parametri, il primo AF_INET specifica che vogliamo un socket che utilizzi IPv4, il secondo SOCK_STREAM specifica che vogliamo una connessione TCP.

s = socket.socket (socket.AF_INET, socket.SOCK_STREAM)

La funzione socket.socket restituisce come valore un nuovo socket, che noi abbiamo chiamato s

IPv4

Connessioni TCP

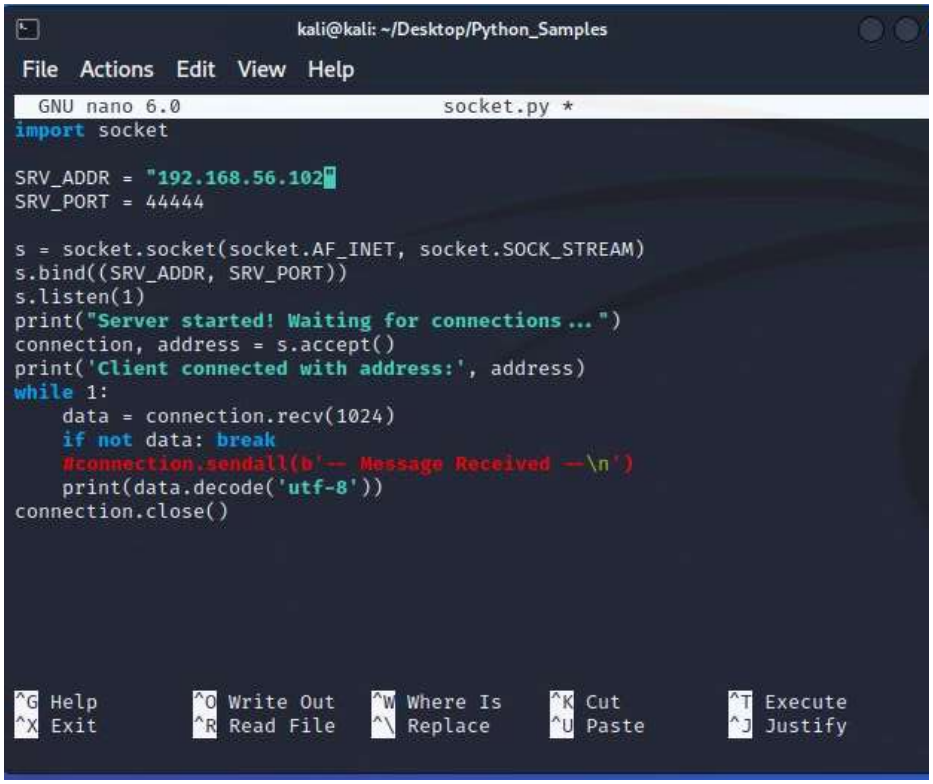


Socket di Rete

4. Dopo aver creato un nuovo socket, dobbiamo collegare il socket all'indirizzo e la porta che abbiamo specificato. Questa azione prende il nome di «binding».
   La sintassi è piuttosto semplice, utilizziamo il metodo «bind», accedendo al nuovo socket con l'operatore «.», così da avere «s.bind». La funzione accetta come parametri, l'indirizzo del server e la porta.

s.bind ((SRV_ADDR , SRV_PORT))

- SRV_ADDR → Metodo bind per associare socket ad IP:PORTA
- SRV_PORT → IP del server
- (SRV_PORT) → Porta dove associare il servizio



Socket di Rete

5. Il metodo seguente, s.listen() configura il socket per ascoltare sulla coppia IP:PORTA che abbiamo indicato.
   Il numero tra parentesi indica il numero massimo di connessioni in coda.

s.listen (backlog)

Configura il socket in ascolto sulla coppia IP:PORTA

Il backlog indica il numero di connessioni max in coda

kali@kali: ~/Desktop/Python_Samples

File Actions Edit View Help

GNU nano 6.0 socket.py *

```
import socket

SRV_ADDR = "192.168.56.102"
SRV_PORT = 44444

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind((SRV_ADDR, SRV_PORT))
s.listen(1)
print("Server started! Waiting for connections ...")
connection, address = s.accept()
print('Client connected with address:', address)
while 1:
    data = connection.recv(1024)
    if not data: break
    #connection.sendall(b'-- Message Received --\n')
    print(data.decode('utf-8'))
connection.close()
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

Socket di Rete

6. Dopo la print (che conosciamo bene), utilizziamo il metodo «accept» per accettare e stabilire la connessione con il client che tenterà di connettersi al nostro servizio. «accept» restituisce due argomenti:
   Connection: l'identificativo della comunicazione che verrà utilizzato per scambiare i dati.
   Address: l'IP del client che si collegherà.

connection, address = s.accept ()

- connection → Oggetto socket che utilizzeremo per scambio dati
- address → IPv4 e porta del client che si connette
- s.accept() → Metodo per accettare connessioni in entrata

kali@kali: ~/Desktop/Python_Samples

File Actions Edit View Help

GNU nano 6.0 socket.py *

```
import socket

SRV_ADDR = "192.168.56.102"
SRV_PORT = 44444

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind((SRV_ADDR, SRV_PORT))
s.listen(1)
print("Server started! Waiting for connections ...")
connection, address = s.accept()
print('Client connected with address:', address)
while 1:
    data = connection.recv(1024)
    if not data: break
    #connection.sendall(b'-- Message Received --\n')
    print(data.decode('utf-8'))
connection.close()
```

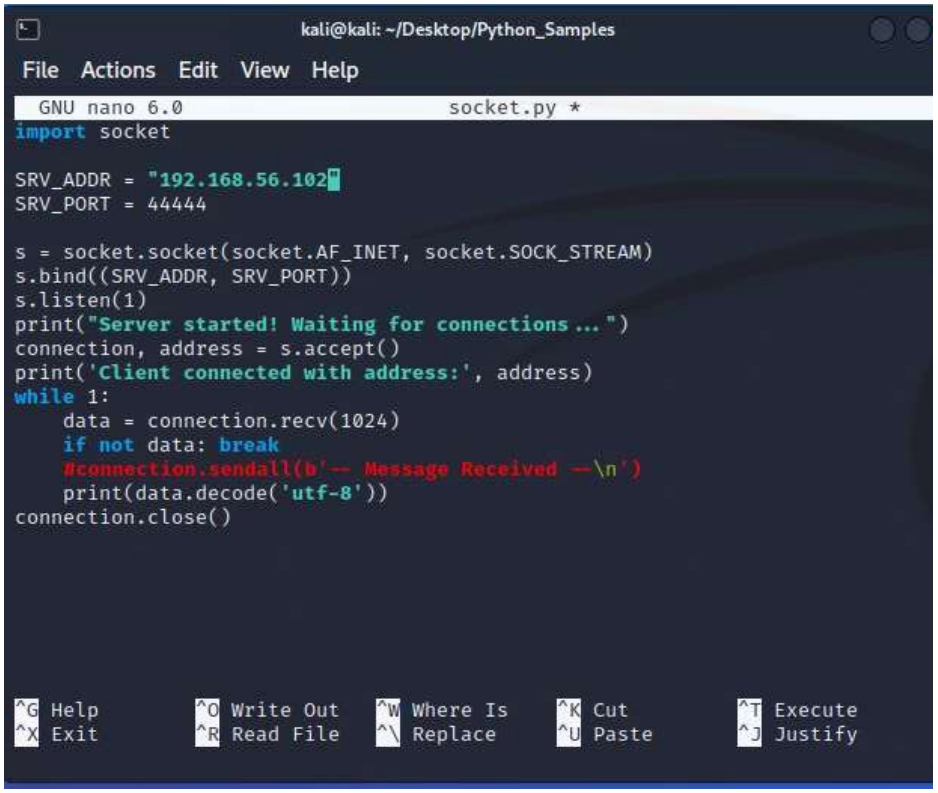
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

Socket di Rete

7. Una volta che il client si è connesso possiamo far partire lo scambio dati. Lo facciamo con un ciclo while sempre VERO (1 in Python equivale a vero, quindi il ciclo while in figura è un ciclo infinito). «connection.recv(1024)» utilizzato per ricevere i dati dal client, mentre la print stampa i dati a schermo dopo averli decodificati nel formato utf-8 (necessario perché il client invia i dati con codifica utf-8).

data = connection.recv (1024)

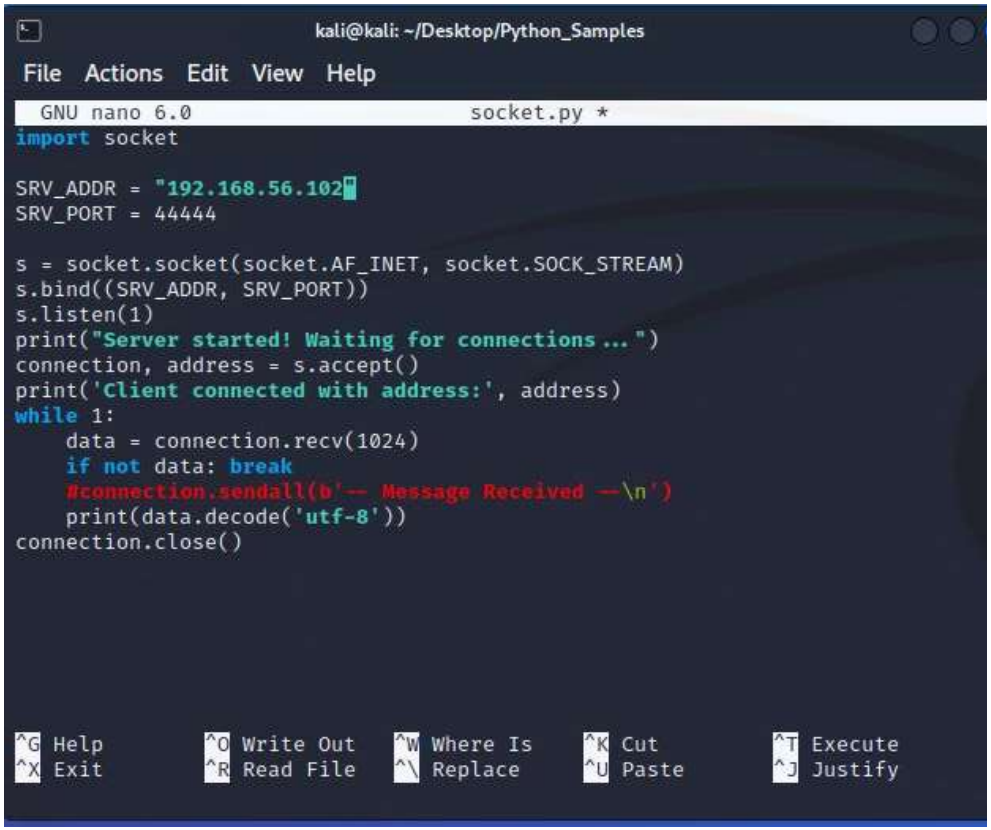
- Contiene i dati ricevuti dal client
- Si utilizza per ricevere i dati dal client
- Grandezza del buffer in byte



Socket di Rete

8. Infine, connection.close() chiude la connessione.

   connection.sendall (commentato) è un comando che serve per inviare a tutti i partecipanti un messaggio che scrive a schermo «messaggio ricevuto», una sorta di conferma che il messaggio è arrivato a destinazione.



Eseguiamo ora il nostro programma e vediamo come connetterci lato client utilizzando un tool di Kali Linux chiamato «netcat». La figura in basso mostra a sinistra il nostro programma in esecuzione, come vedete è inizialmente in ascolto, poi accetta la connessione del client. Lato client usiamo il comando «netcat IP PORTA» per connetterci. Una volta connessi possiamo inviare i messaggi dal client, che come vedete vengono ricevuti dal server. Notate che quando il client si connette, il server ne recupera l'indirizzo IP e la porta sorgente dalla quale il cliente fa partire la connessione verso il server sulla porta 44444.

EPICODE

Port Scanner

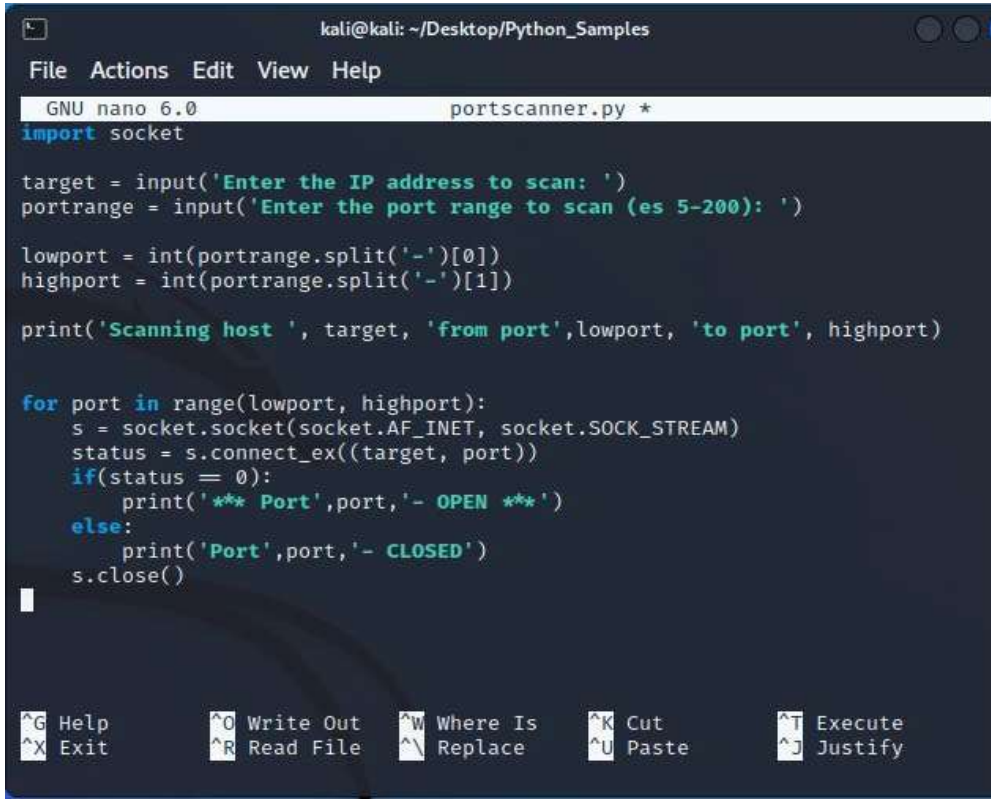
Port-Scanner

Il port-scanner, come suggerisce la parola, prende in input un IP ed un intervallo di porte e controlla se queste sono aperte o meno. Per un Hacker è fondamentale sapere quali porte sono aperte e quali servizi sono attivi. Vediamo il codice:

1. Partiamo con l'analisi. La prima cosa che notiamo è che anche in questo caso importiamo le funzioni del modulo socket tramite l'import in prima riga.

2. Il programma chiede all'utente di inserire un IP ed un range di porte da scansionare, in formato standard esempio 1-1024.

3. Lavoriamo sulla stringa inserita dall'utente per ricavare il range, e inseriamo gli estremi nelle variabili «lowport» e «highport». Notate che per lavorare sulla stringa, abbiamo utilizzato il metodo «**split**» e utilizzato il simbolo «**-**» come separatore.



Port-Scanner

4. Il ciclo «for» successivo ci serve per tentare la connessione TCP ad ogni porta nell'intervallo specificato. Utilizziamo il metodo «socket.socket» che già conosciamo, che ci restituisce un socket che chiamiamo «s», notate che abbiamo scelto come parametri della socket, IPv4 e TCP.

5. La funzione successiva, «s.connect_ex» tenta la connessione alla coppia IP:PORTA specificata, e ci restituisce uno stato che può essere:
   - «0»: la connessione è andata a buon fine, dunque possiamo dedurre che la porta è aperta.
   - «diverso da 0»: la connessione non è andata a buon fine, dunque possiamo dire che la porta non è aperta.

kali@kali: ~/Desktop/Python_Samples

File Actions Edit View Help

GNU nano 6.0 portscanner.py *

```
import socket

target = input('Enter the IP address to scan: ')
portrange = input('Enter the port range to scan (es 5-200): ')

lowport = int(portrange.split('-')[0])
highport = int(portrange.split('-')[1])

print('Scanning host ', target, 'from port',lowport, 'to port', highport)


for port in range(lowport, highport):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    status = s.connect_ex((target, port))
    if(status == 0):
        print('** Port',port,'- OPEN **')
    else:
        print('Port',port,'- CLOSED')
    s.close()
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

EPICODE

Rilevatore verbi HTTP

Rilevatore di Verbi HTTP

Con il prossimo esempio di codice, vediamo come si possono recuperare informazioni circa i verbi HTTP che sono abilitati su un determinato Web Server remoto. Per quest'esempio si utilizzerà il modulo http.client. Diamo uno sguardo al codice ed analizziamolo insieme:

1. La prima parte del programma è piuttosto nota, il programma carica il modulo http.client per utilizzarne le funzioni, successivamente chiede all'utente di inserire:

   - Host / indirizzo IP del sistema target.
   - La porta. Se la porta è vuota, verrà utilizzata la porta di default 80.

kali@kali: ~/Desktop/Python_Samples

File Actions Edit View Help

GNU nano 6.0 httpstatus.py

```
import http.client

host = input("Inserire  host/IP del sistema target: ")
port = input("Insert la porta del sistema target (default:80): ")

if(port == ""):
    port = 80

try:
    connection = http.client.HTTPConnection(host, port)
    connection.request('OPTIONS', '/')
    response = connection.getresponse()
    print("I metodi abilitati sono:",response.status)
    connection.close()
except ConnectionRefusedError:
    print("Connessione fallita")
```

Rilevatore di Verbi HTTP

2. La seconda parte gestisce la connessione verso l'host e lo fa tramite la funzione «http.client.HTTPConnection» che prende in input i parametri «host, porta». La funzione restituisce l'oggetto «connection».

3. Il metodo «request», è utilizzato per inviare una richiesta http specificando verbo, e path. In questo caso utilizziamo «option» come verbo, e la root directory «/» come path.

kali@kali: ~/Desktop/Python_Samples

File Actions Edit View Help

GNU nano 6.0 httpstatus.py

```python
import http.client

host = input("Inserire  host/IP del sistema target: ")
port = input("Insert la porta del sistema target (default:80): ")

if(port == ""):
    port = 80

try:
    connection = http.client.HTTPConnection(host, port)
    connection.request('OPTIONS', '/')
    response = connection.getresponse()
    print("I metodi abilitati sono:",response.status)
    connection.close()
except ConnectionRefusedError:
    print("Connessione fallita")
```

Rilevatore di Verbi HTTP

4. La risposta del server è prima salvata nella variabile response, poi tramite il metodo «status», si scrivono a schermo i verbi http abilitati in base alla risposta del server.

kali@kali: ~/Desktop/Python_Samples

File Actions Edit View Help

GNU nano 6.0 httpstatus.py

```
import http.client

host = input("Inserire  host/IP del sistema target: ")
port = input("Insert la porta del sistema target (default:80): ")

if(port == ""):
    port = 80

try:
    connection = http.client.HTTPConnection(host, port)
    connection.request('OPTIONS', '/')
    response = connection.getresponse()
    print("I metodi abilitati sono:",response.status)
    connection.close()
except ConnectionRefusedError:
    print("Connessione fallita")
```

Rilevatore di Verbi HTTP

Notate che nell'esempio a destra abbiamo scelto di inviare al server una HTTP request «OPTIONS». Volendo avremmo potuto utilizzare uno degli altri verbi messi a disposizione come GET, POST, PUT.
I parametri sono diversi in base al verbo che si sceglie. Per esempio «POST» è spesso usato per inviare informazioni ad web server, per esempio per una login.

kali@kali: ~/Desktop/Python_Samples

File Actions Edit View Help

GNU nano 6.0 httpstatus.py

```
import http.client

host = input("Inserire  host/IP del sistema target: ")
port = input("Insert la porta del sistema target (default:80): ")

if(port == ""):
    port = 80

try:
    connection = http.client.HTTPConnection(host, port)
    connection.request('OPTIONS', '/')
    response = connection.getresponse()
    print("I metodi abilitati sono:",response.status)
    connection.close()
except ConnectionRefusedError:
    print("Connessione fallita")
```

EPICODE

Esercizi Facili

Esercizi Facili

1. **Calcolo della somma di due numeri**
   - Scrivi un programma che chieda all'utente di inserire due numeri e poi calcoli e mostri la loro somma.
2. **Verifica di un numero pari o dispari**
   - Scrivi un programma che chieda all'utente di inserire un numero e determini se è pari o dispari
3. **Contare il numero di caratteri in una stringa**
   - Scrivi un programma che chieda all'utente di inserire una stringa e poi conti e mostri il numero di caratteri in essa.
4. **Verifica di un anno bisestile**
   - Scrivi un programma che chieda all'utente di inserire un anno e determini se è bisestile.

Esercizi Facili

1. **Calcolo della somma di due numeri**
   - Scrivi un programma che chieda all'utente di inserire due numeri e poi calcoli e mostri la loro somma.

```
numero1 = int(input("Inserisci il primo numero: "))
numero2 = int(input("Inserisci il secondo numero: "))
somma = numero1 + numero2
print(f"La somma dei due numeri è: {somma}")
```

1. **Verifica di un numero pari o dispari**
   - Scrivi un programma che chieda all'utente di inserire un numero e determini se è pari o dispari.

```
numero = int(input("Inserisci un numero: "))
if numero % 2 == 0:
  print("Il numero è pari")
else:
  print("Il numero è dispari")
```

Esercizi Facili

3. **Contare il numero di caratteri in una stringa**
   - Scrivi un programma che chieda all'utente di inserire una stringa e poi conti e mostri il numero di caratteri in essa.

```
stringa = input("Inserisci una stringa: ")
print(f"Il numero di caratteri nella stringa è: {len(stringa)}")
```

3. **Verifica di un anno bisestile**
   - Scrivi un programma che chieda all'utente di inserire un anno e determini se è bisestile.

```
anno = int(input("Inserisci un anno: "))
if (anno % 4 == 0 and anno % 100 != 0) or (anno % 400 == 0):
  print(f"{anno} è un anno bisestile")
else:
  print(f"{anno} non è un anno bisestile")
```

EPICODE

Esercizi Media Difficoltà

Esercizi di Media Difficoltà

1. **Calcolo del fattoriale di un numero**
   - Scrivi un programma che chieda all'utente di inserire un numero intero e calcoli il suo fattoriale
2. **Generare la sequenza di Fibonacci**
   - Scrivi un programma che chieda all'utente di inserire un numero n e generi i primi n numeri della sequenza di Fibonacci.
3. **Invertire una stringa**
   - Scrivi un programma che chieda all'utente di inserire una stringa e poi la inverta.

EPICODE

Esercizi Difficili

Esercizi Difficili

1. **Conteggio delle occorrenze di ogni parola in una stringa**
   - Scrivi un programma che chieda all'utente di inserire una stringa e poi conti le occorrenze di ogni parola in essa.
2. **Calcolo della media, mediana e moda di una lista di numeri**
   - Scrivi un programma che chieda all'utente di inserire una lista di numeri (separati da spazi) e calcoli la media, la mediana e la moda.

EPICODE

GRAZIE

Epicode